78ca23cc-a05e-4e26-a0c6-95708748277f

<prev_user>

<request>

You are working on an issue in the 'jamiebull1/geomeppy' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/geomeppy/geomeppy (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-03-13T00:04:21.282Z</current_datetime>

Consider the following problem statement:

<problem_statement>
There's a mypy error: geomeppy/patches.py:295: error: Expected an indented block after function definition on line 39 [syntax]
Found 1 error in 1 file (errors prevented further checking)
Error: Process completed with exit code 2.
</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

All issues are resolved. Here's a summary of what was fixed:

Changes Made

`geomeppy/patches.py`
1. **Line 295 (syntax error root cause)**: Removed `# type: Dict[str, Idf_MSequence]` annotation from `self.idfobjects[key].append(abunch)`. A `# type:` comment on a non-assignment statement (a function call) is invalid per PEP 484 and caused mypy's parser to fail with "Expected an indented block after function definition on line 39".

2. **Line 72**: Changed `commdct=None,  # type: List[List[Dict[str, Any]]]` → `Optional[List[List[Dict[str, Any]]]]` since the default value `None` was incompatible with the non-Optional type annotation (previously hidden behind the syntax error).

`geomeppy/idf.py`
3. Added `Iterable` to the `from typing import ...` line and annotated the three `surfaces` local variables as `# type: Iterable[Any]`. This allows the variable to be reassigned from `itertools.chain` to `filter` without a type incompatibility error (both are `Iterable`).

Security summary: No security vulnerabilities were introduced or discovered. All changes are type annotation fixes only.

<current>

<request>
You are working on an issue in the 'jamiebull1/geomeppy' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/geomeppy/geomeppy (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-03-13T00:15:27.381Z</current_datetime>

Consider the following problem statement:

<problem_statement>
There are still mypy errors... 7s
Run mypy --ignore-missing-imports geomeppy
geomeppy/geom/vectors.py:67: error: No overload variant of "dot" matches argument types "Vector2D", "Vector3D" [call-overload]
geomeppy/geom/vectors.py:67: note: Possible overload variants:
geomeppy/geom/vectors.py:67: note: def dot(a: _Buffer | _SupportsArray[dtype[Any]] | _NestedSequence[_SupportsArray[dtype[Any]]] | complex | bytes |

```
str | _NestedSequence[complex | bytes | str], b: _Buffer |
_SupportsArray[dtype[Any]] |
_NestedSequence[_SupportsArray[dtype[Any]]] | complex | bytes |
str | _NestedSequence[complex | bytes | str], out: None = ...)
-&gt; Any
geomeppy/geom/vectors.py:67: note:     def [_ArrayT:
ndarray[tuple[Any, ...], dtype[Any]]] dot(a: _Buffer |
_SupportsArray[dtype[Any]] |
_NestedSequence[_SupportsArray[dtype[Any]]] | complex | bytes |
str | _NestedSequence[complex | bytes | str], b: _Buffer |
_SupportsArray[dtype[Any]] |
_NestedSequence[_SupportsArray[dtype[Any]]] | complex | bytes |
str | _NestedSequence[complex | bytes | str], out: _ArrayT)
-&gt; _ArrayT
geomeppy/geom/vectors.py:71: error: No overload variant of
"cross" matches argument types "Vector2D", "Vector2D |
Vector3D"  [call-overload]
geomeppy/geom/vectors.py:71: note: Possible overload variants:
geomeppy/geom/vectors.py:71: note:     def [_AnyNumericScalarT:
(signedinteger[_8Bit], signedinteger[_16Bit],
signedinteger[_32Bit], signedinteger[_64Bit],
unsignedinteger[_8Bit], unsignedinteger[_16Bit],
unsignedinteger[_32Bit], unsignedinteger[_64Bit],
floating[_16Bit], floating[_32Bit], float64, floating[_64Bit |
_96Bit | _128Bit], complexfloating[_32Bit, _32Bit], complex128,
complexfloating[_64Bit | _96Bit | _128Bit, _64Bit | _96Bit |
_128Bit], timedelta64[timedelta | int | None], object_)]
cross(a: _SupportsArray[dtype[_AnyNumericScalarT]] |
_NestedSequence[_SupportsArray[dtype[_AnyNumericScalarT]]], b:
_SupportsArray[dtype[_AnyNumericScalarT]] |
_NestedSequence[_SupportsArray[dtype[_AnyNumericScalarT]]],
axisa: int = ..., axisb: int = ..., axisc: int = ..., axis: int
| None = ...) -&gt; ndarray[tuple[Any, ...],
dtype[_AnyNumericScalarT]]
geomeppy/geom/vectors.py:71: note:     def cross(a:
_SupportsArray[dtype[numpy.bool[builtins.bool] | integer[Any]]]
| _NestedSequence[_SupportsArray[dtype[numpy.bool[builtins.bool]
| integer[Any]]]] | int | _NestedSequence[int], b:
_SupportsArray[dtype[numpy.bool[builtins.bool] | integer[Any]]]
| _NestedSequence[_SupportsArray[dtype[numpy.bool[builtins.bool]
| integer[Any]]]] | int | _NestedSequence[int], axisa: int
= ..., axisb: int = ..., axisc: int = ..., axis: int | None
= ...) -&gt; ndarray[tuple[Any, ...], dtype[signedinteger[_32Bit
| _64Bit] | Any]]
geomeppy/geom/vectors.py:71: note:     def cross(a:
_SupportsArray[dtype[numpy.bool[builtins.bool] | integer[Any] |
floating[Any]]] |
```

```
_NestedSequence[_SupportsArray[dtype[numpy.bool[builtins.bool] |
integer[Any] | floating[Any]]]] | float |
_NestedSequence[float], b:
_SupportsArray[dtype[numpy.bool[builtins.bool] | integer[Any] |
floating[Any]]] |
_NestedSequence[_SupportsArray[dtype[numpy.bool[builtins.bool] |
integer[Any] | floating[Any]]]] | float |
_NestedSequence[float], axisa: int = ..., axisb: int = ...,
axisc: int = ..., axis: int | None = ...) -&gt;
ndarray[tuple[Any, ...], dtype[float64 | Any]]
geomeppy/geom/vectors.py:71: note:     def cross(a:
_SupportsArray[dtype[numpy.bool[builtins.bool] | number[Any, int
| float | complex]]] |
_NestedSequence[_SupportsArray[dtype[numpy.bool[builtins.bool] |
number[Any, int | float | complex]]]] | complex |
_NestedSequence[complex], b:
_SupportsArray[dtype[numpy.bool[builtins.bool] | number[Any, int
| float | complex]]] |
_NestedSequence[_SupportsArray[dtype[numpy.bool[builtins.bool] |
number[Any, int | float | complex]]]] | complex |
_NestedSequence[complex], axisa: int = ..., axisb: int = ...,
axisc: int = ..., axis: int | None = ...) -&gt;
ndarray[tuple[Any, ...], dtype[complex128 | Any]]
geomeppy/geom/transformations.py:66: error: No overload variant
of "__mul__" of "float64" matches argument type "Vector2D"
[operator]
geomeppy/geom/transformations.py:66: note: Possible overload
variants:
geomeppy/geom/transformations.py:66: note:     def __mul__(self,
float | floating[_64Bit] | floating[_32Bit] | floating[_16Bit] |
integer[Any] | numpy.bool[builtins.bool], /) -&gt; float64
geomeppy/geom/transformations.py:66: note:     def __mul__(self,
complexfloating[_64Bit, _64Bit], /) -&gt; complex128
geomeppy/geom/transformations.py:66: note:     def [_NBit1:
NBitBase, _NBit2: NBitBase] __mul__(self,
complexfloating[_NBit1, _NBit2], /) -&gt; complexfloating[_NBit1
| _64Bit, _NBit2 | _64Bit]
geomeppy/geom/transformations.py:66: note:     def __mul__(self,
complex, /) -&gt; float64 | complex128
geomeppy/geom/transformations.py:71: error: No overload variant
of "__mul__" of "float64" matches argument type "Vector2D"
[operator]
geomeppy/geom/transformations.py:71: note: Possible overload
variants:
geomeppy/geom/transformations.py:71: note:     def __mul__(self,
float | floating[_64Bit] | floating[_32Bit] | floating[_16Bit] |
integer[Any] | numpy.bool[builtins.bool], /) -&gt; float64
```

```
geomeppy/geom/transformations.py:71: note:     def __mul__(self,
complexfloating[_64Bit, _64Bit], /) -&gt; complex128
geomeppy/geom/transformations.py:71: note:     def [_NBit1:
NBitBase, _NBit2: NBitBase] __mul__(self,
complexfloating[_NBit1, _NBit2], /) -&gt; complexfloating[_NBit1
| _64Bit, _NBit2 | _64Bit]
geomeppy/geom/transformations.py:71: note:     def __mul__(self,
complex, /) -&gt; float64 | complex128
geomeppy/geom/transformations.py:123: error: Name "np.float" is
not defined  [name-defined]
geomeppy/geom/polygons.py:540: error: No overload variant of
"__truediv__" of "float64" matches argument type "list[Any]"
[operator]
geomeppy/geom/polygons.py:540: note: Possible overload variants:
geomeppy/geom/polygons.py:540: note:     def __truediv__(self,
float | floating[_64Bit] | floating[_32Bit] | floating[_16Bit] |
integer[Any] | numpy.bool[builtins.bool], /) -&gt; float64
geomeppy/geom/polygons.py:540: note:     def __truediv__(self,
complexfloating[_64Bit, _64Bit], /) -&gt; complex128
geomeppy/geom/polygons.py:540: note:     def [_NBit1: NBitBase,
_NBit2: NBitBase] __truediv__(self, complexfloating[_NBit1,
_NBit2], /) -&gt; complexfloating[_NBit1 | _64Bit, _NBit2 |
_64Bit]
geomeppy/geom/polygons.py:540: note:     def __truediv__(self,
complex, /) -&gt; float64 | complex128
geomeppy/geom/polygons.py:540: note: Right operand is of type
"Any | list[Any]"
geomeppy/builder.py:210: note: By default the bodies of untyped
functions are not checked, consider using --check-untyped-defs
[annotation-unchecked]
geomeppy/view_geometry.py:53: error: "Axes" has no attribute
"add_collection3d"  [attr-defined]
geomeppy/view_geometry.py:59: error: "Axes" has no attribute
"set_zlim"  [attr-defined]
Found 8 errors in 4 files (checked 19 source files)
Error: Process completed with exit code 1.
</problem_statement>
```

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prompt>

```
There are still mypy errors... 7s
Run mypy --ignore-missing-imports geomeppy
geomeppy/geom/vectors.py:67: error: No overload variant of "dot"
matches argument types "Vector2D", "Vector3D"  [call-overload]
geomeppy/geom/vectors.py:67: note: Possible overload variants:
geomeppy/geom/vectors.py:67: note:     def dot(a: _Buffer |
_SupportsArray[dtype[Any]] |
_NestedSequence[_SupportsArray[dtype[Any]]] | complex | bytes | str
| _NestedSequence[complex | bytes | str], b: _Buffer |
_SupportsArray[dtype[Any]] |
_NestedSequence[_SupportsArray[dtype[Any]]] | complex | bytes | str
| _NestedSequence[complex | bytes | str], out: None = ...) -> Any
geomeppy/geom/vectors.py:67: note:     def [_ArrayT:
ndarray[tuple[Any, ...], dtype[Any]]] dot(a: _Buffer |
_SupportsArray[dtype[Any]] |
_NestedSequence[_SupportsArray[dtype[Any]]] | complex | bytes | str
| _NestedSequence[complex | bytes | str], b: _Buffer |
_SupportsArray[dtype[Any]] |
_NestedSequence[_SupportsArray[dtype[Any]]] | complex | bytes | str
| _NestedSequence[complex | bytes | str], out: _ArrayT) -> _ArrayT
geomeppy/geom/vectors.py:71: error: No overload variant of "cross"
matches argument types "Vector2D", "Vector2D | Vector3D"  [call-
overload]
geomeppy/geom/vectors.py:71: note: Possible overload variants:
geomeppy/geom/vectors.py:71: note:     def [_AnyNumericScalarT:
(signedinteger[_8Bit], signedinteger[_16Bit], signedinteger[_32Bit],
signedinteger[_64Bit], unsignedinteger[_8Bit],
unsignedinteger[_16Bit], unsignedinteger[_32Bit],
unsignedinteger[_64Bit], floating[_16Bit], floating[_32Bit],
float64, floating[_64Bit | _96Bit | _128Bit],
complexfloating[_32Bit, _32Bit], complex128, complexfloating[_64Bit
| _96Bit | _128Bit, _64Bit | _96Bit | _128Bit],
timedelta64[timedelta | int | None], object_)] cross(a:
_SupportsArray[dtype[_AnyNumericScalarT]] |
_NestedSequence[_SupportsArray[dtype[_AnyNumericScalarT]]], b:
_SupportsArray[dtype[_AnyNumericScalarT]] |
_NestedSequence[_SupportsArray[dtype[_AnyNumericScalarT]]], axisa:
int = ..., axisb: int = ..., axisc: int = ..., axis: int | None
= ...) -> ndarray[tuple[Any, ...], dtype[_AnyNumericScalarT]]
geomeppy/geom/vectors.py:71: note:     def cross(a:
_SupportsArray[dtype[numpy.bool[builtins.bool] | integer[Any]]] |
_NestedSequence[_SupportsArray[dtype[numpy.bool[builtins.bool] |
integer[Any]]]] | int | _NestedSequence[int], b:
_SupportsArray[dtype[numpy.bool[builtins.bool] | integer[Any]]] |
_NestedSequence[_SupportsArray[dtype[numpy.bool[builtins.bool] |
```

```
integer[Any]]]] | int | _NestedSequence[int], axisa: int = ...,
axisb: int = ..., axisc: int = ..., axis: int | None = ...) ->
ndarray[tuple[Any, ...], dtype[signedinteger[_32Bit | _64Bit] |
Any]]
geomeppy/geom/vectors.py:71: note:     def cross(a:
_SupportsArray[dtype[numpy.bool[builtins.bool] | integer[Any] |
floating[Any]]] |
_NestedSequence[_SupportsArray[dtype[numpy.bool[builtins.bool] |
integer[Any] | floating[Any]]]] | float | _NestedSequence[float], b:
_SupportsArray[dtype[numpy.bool[builtins.bool] | integer[Any] |
floating[Any]]] |
_NestedSequence[_SupportsArray[dtype[numpy.bool[builtins.bool] |
integer[Any] | floating[Any]]]] | float | _NestedSequence[float],
axisa: int = ..., axisb: int = ..., axisc: int = ..., axis: int |
None = ...) -> ndarray[tuple[Any, ...], dtype[float64 | Any]]
geomeppy/geom/vectors.py:71: note:     def cross(a:
_SupportsArray[dtype[numpy.bool[builtins.bool] | number[Any, int |
float | complex]]] |
_NestedSequence[_SupportsArray[dtype[numpy.bool[builtins.bool] |
number[Any, int | float | complex]]]] | complex |
_NestedSequence[complex], b:
_SupportsArray[dtype[numpy.bool[builtins.bool] | number[Any, int |
float | complex]]] |
_NestedSequence[_SupportsArray[dtype[numpy.bool[builtins.bool] |
number[Any, int | float | complex]]]] | complex |
_NestedSequence[complex], axisa: int = ..., axisb: int = ..., axisc:
int = ..., axis: int | None = ...) -> ndarray[tuple[Any, ...],
dtype[complex128 | Any]]
geomeppy/geom/transformations.py:66: error: No overload variant of
"__mul__" of "float64" matches argument type "Vector2D"  [operator]
geomeppy/geom/transformations.py:66: note: Possible overload
variants:
geomeppy/geom/transformations.py:66: note:     def __mul__(self,
float | floating[_64Bit] | floating[_32Bit] | floating[_16Bit] |
integer[Any] | numpy.bool[builtins.bool], /) -> float64
geomeppy/geom/transformations.py:66: note:     def __mul__(self,
complexfloating[_64Bit, _64Bit], /) -> complex128
geomeppy/geom/transformations.py:66: note:     def [_NBit1:
NBitBase, _NBit2: NBitBase] __mul__(self, complexfloating[_NBit1,
_NBit2], /) -> complexfloating[_NBit1 | _64Bit, _NBit2 | _64Bit]
geomeppy/geom/transformations.py:66: note:     def __mul__(self,
complex, /) -> float64 | complex128
geomeppy/geom/transformations.py:71: error: No overload variant of
"__mul__" of "float64" matches argument type "Vector2D"  [operator]
geomeppy/geom/transformations.py:71: note: Possible overload
variants:
geomeppy/geom/transformations.py:71: note:     def __mul__(self,
```

```
float | floating[_64Bit] | floating[_32Bit] | floating[_16Bit] |
integer[Any] | numpy.bool[builtins.bool], /) -> float64
geomeppy/geom/transformations.py:71: note:     def __mul__(self,
complexfloating[_64Bit, _64Bit], /) -> complex128
geomeppy/geom/transformations.py:71: note:     def [_NBit1:
NBitBase, _NBit2: NBitBase] __mul__(self, complexfloating[_NBit1,
_NBit2], /) -> complexfloating[_NBit1 | _64Bit, _NBit2 | _64Bit]
geomeppy/geom/transformations.py:71: note:     def __mul__(self,
complex, /) -> float64 | complex128
geomeppy/geom/transformations.py:123: error: Name "np.float" is not
defined  [name-defined]
geomeppy/geom/polygons.py:540: error: No overload variant of
"__truediv__" of "float64" matches argument type "list[Any]"
[operator]
geomeppy/geom/polygons.py:540: note: Possible overload variants:
geomeppy/geom/polygons.py:540: note:     def __truediv__(self, float
| floating[_64Bit] | floating[_32Bit] | floating[_16Bit] |
integer[Any] | numpy.bool[builtins.bool], /) -> float64
geomeppy/geom/polygons.py:540: note:     def __truediv__(self,
complexfloating[_64Bit, _64Bit], /) -> complex128
geomeppy/geom/polygons.py:540: note:     def [_NBit1: NBitBase,
_NBit2: NBitBase] __truediv__(self, complexfloating[_NBit1,
_NBit2], /) -> complexfloating[_NBit1 | _64Bit, _NBit2 | _64Bit]
geomeppy/geom/polygons.py:540: note:     def __truediv__(self,
complex, /) -> float64 | complex128
geomeppy/geom/polygons.py:540: note: Right operand is of type "Any |
list[Any]"
geomeppy/builder.py:210: note: By default the bodies of untyped
functions are not checked, consider using --check-untyped-defs
[annotation-unchecked]
geomeppy/view_geometry.py:53: error: "Axes" has no attribute
"add_collection3d"  [attr-defined]
geomeppy/view_geometry.py:59: error: "Axes" has no attribute
"set_zlim"  [attr-defined]
Found 8 errors in 4 files (checked 19 source files)
Error: Process completed with exit code 1.
```